Algorithms

Shortest Path Algorithms (cont')

Reading: CLRS, Sections 24, 25.2



Review – Lecture 5

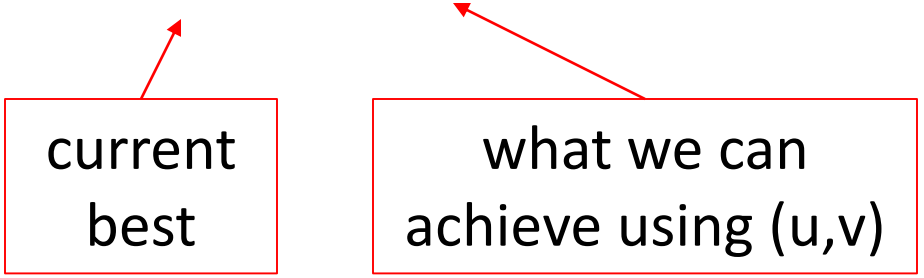
Shortest path calculation:

General structure of a single source algorithm:

- For every node $v \in V$:

v.d = length of shortest path detected so far.

- v.d is initialized to infinity (except s.d = 0).
- The value of v.d may be updated (decreased) in relax operations.
- relax(u,v): $v.d = \min\{v.d, u.d+w(u,v)\}$



Review – Lecture 5

- Algorithm for DAG (arbitrary weights)

1. Topological sort.
2. Consider the vertices according to the toposort. When v is considered, relax all the edges outgoing from v.

- Time complexity O(n+m)

Review – Lecture 5

- Algorithm for graphs with non-negative weights – Dijkstra
  - Use a data structure Q that supports 'find-min' operation. Store the v.d values in Q.
  - In every iteration, consider a node having minimal v.d value. When v is considered relax all the edges outgoing from v, and remove v from Q.
- Three different implementations. O((m+n)logn) or $O(n^2)$ or O(m + n log n)

Review – Lecture 5

- Things the two algorithms have in common:
  - They consider the nodes in some order.
  - When v is considered - relax all the edges outgoing from v.
  - Therefore: Every edge is relaxed once.

In a DAG: The order is known in advance (determined by the toposort).

In Dijkstra: The order is determined dynamically, as it depends on the distance from s.

In both: For every shortest path p, the edges of p are relaxed according to their order in p

Review – Lecture 5

- Bellman Ford Algorithm: For graphs with negative weights.
- Perform at most n-1 sweeps.
- A sweep: relax all edges one after the other.
- Until an entire sweep with no v.d update, or n-1 sweeps.

- Time complexity: O(mn)

- Main idea in the proof: If the shortest path from s to v consists of k edges, then after at most k sweeps, v.d = d(s,v).

Detecting Negative cycles

A negative cycle: A directed cycle whose total edge-weight is negative.

- If a negative cycle exists, then it is always beneficial to repeat it, therefore, the shortest path is infinite and not well-defined.
- Goal: Given a directed graph, identify whether a negative cycle exists.
- Idea: A negative cycle exists iff some shortest path is not simple.
- Implementing this idea: Add to BF algorithm another sweep (sweep number n).

Detecting Negative cycles

- Using BF to detect negative cycles:

Theorem: The graph includes a negative cycle if and only if for some vertex v, v.d is reduced in sweep number n.

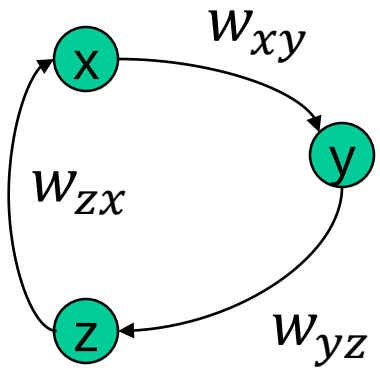
- Practically: After n-1 sweeps perform:
- for each $(u,v) \in E$
  - If $v.d > u.d + w(u,v)$ then print "A negative cycle exists", HALT

Proof: In class + Moodle.

Detecting Negative cycles

- No negative cycle $\Rightarrow$ No print.
  No negative cycle, therefore all SPs are simple. A simple path consists of at most n-1 edges. By the proof of BF, the v.d values are already finalized after n-1 iterations.
- No print $\Rightarrow$ No negative cycle.

  Consider a cycle. Since nothing is printed, for each of the cycle edges, v.d ≤ u.d + w(u,v). (cont' in class)



All-pair Shortest Path

- Input: a directed graph G=(V,E).

  A weight function w: E→ R

  w(u,v) may be negative.
- Let $V = \{1, 2, ..., n\}$.
- Output: All-pair shortest path: for any two vertices v,u in V, what is the shortest path from v to u
  - we will only be interested in the *length* of that path.

All-pair Shortest Path

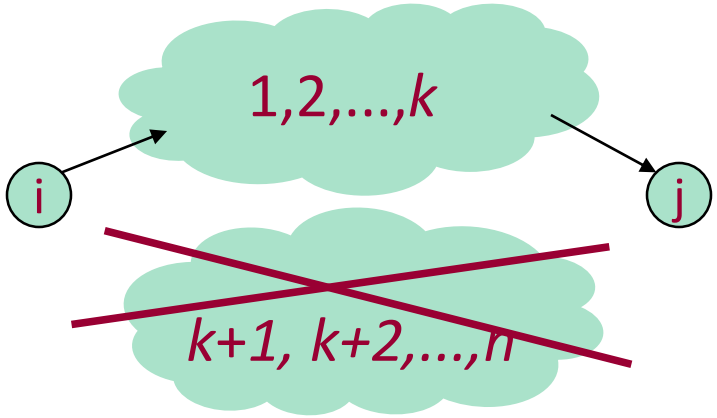
- We can solve this problem using single-source shortest path algorithms. For example, we can run Bellman-Ford |V| times (one time for each possible selection of the source vertex s).
- Time complexity: $|V|*O(|V||E|)=O(|V|^2|E|)$
- We will see a solution based on Dynamic Programming.

All-pair Shortest Path

Define

$$\delta^0(i,j) = \begin{cases} w(e) & \text{if } (i,j) \in E, \\ \infty & \text{if } (i,j) \notin E. \\ 0 & \text{if } i = j \end{cases}$$

Let $\delta^k(i,j)$ be the length of a shortest path from i to j among all paths which may pass through vertices 1,2,...,*k* but do not pass through vertices *k+1, k+2,...,n.*



Floyd -Warshall Algorithm (1962)

1. Init $\delta^0(i, j)$ as defined earlier
2. $k \leftarrow 1$
3. For every $1 \leq i, j \leq n$ compute

$$\delta^k(i, j) \leftarrow \text{Min} \{ \delta^{k-1}(i, j), \delta^{k-1}(i, k) + \delta^{k-1}(k, j)\}.$$

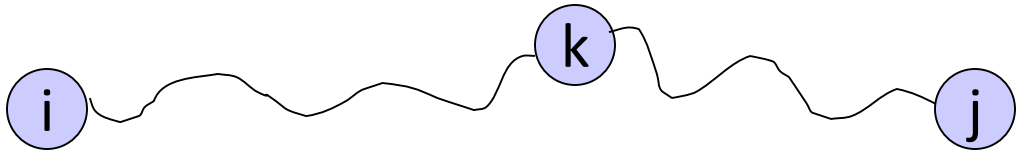
4. If $k = n$, stop. If not, increment k and go to step 3.

Floyd -Warshall Algorithm

$$\delta^k(i, j) \leftarrow \text{Min} \{ \delta^{k-1}(i, j), \delta^{k-1}(i, k) + \delta^{k-1}(k, j)\}.$$

The shortest path from i to j which may pass through vertices 1,2,...,*k* but do not pass through vertices *k+1, k+2,...,n:*

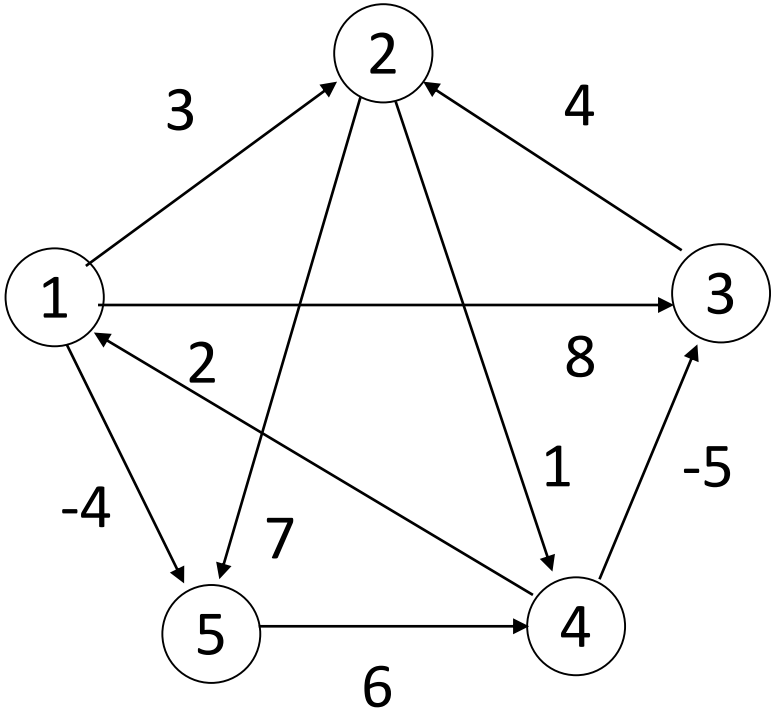
1. Might not pass through vertex k, or
2. Might pass through k, and then it is composed by two already-computed shortest paths.



Floyd -Warshall Algorithm

δ^0

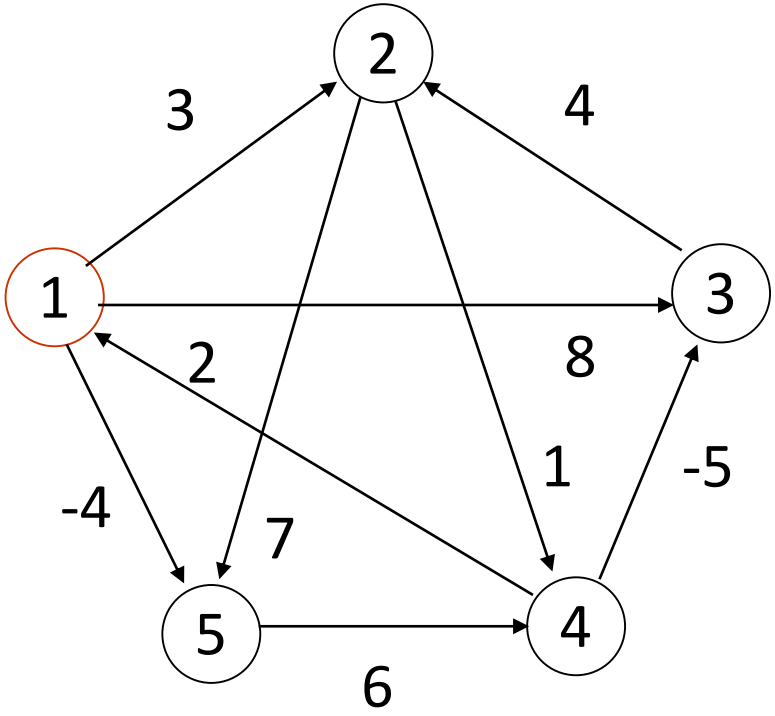
	1	2	3	4	5
1	0	3	8	--	-4
2	--	0	--	1	7
3	--	4	0	--	--
4	2	--	-5	0	--
5	--	--	--	6	0



Floyd-Warshall Algorithm

δ^1

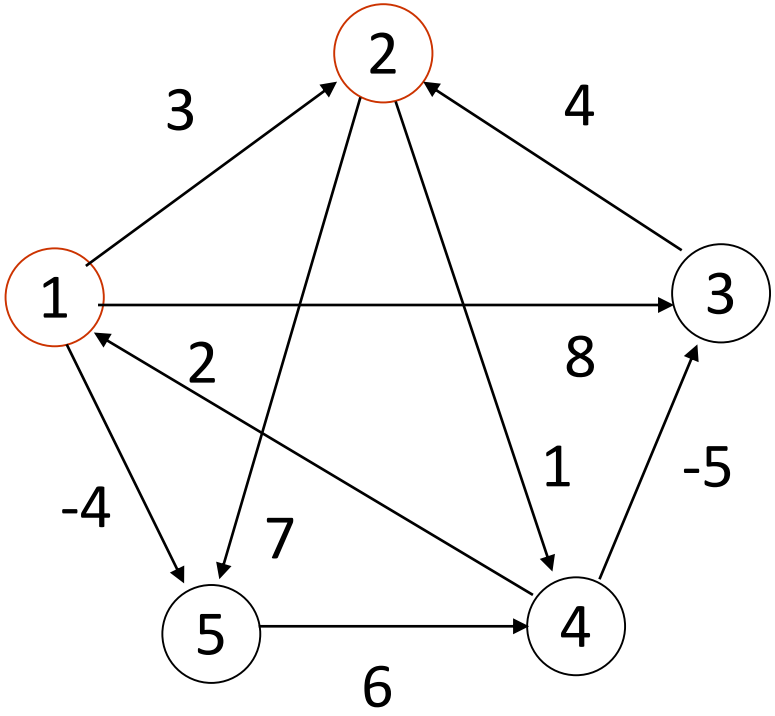
	1	2	3	4	5
1	0	3	8	--	-4
2	--	0	--	1	7
3	--	4	0	--	--
4	2	5	-5	0	-2
5	--	--	--	6	0



Floyd -Warshall Algorithm

δ^2

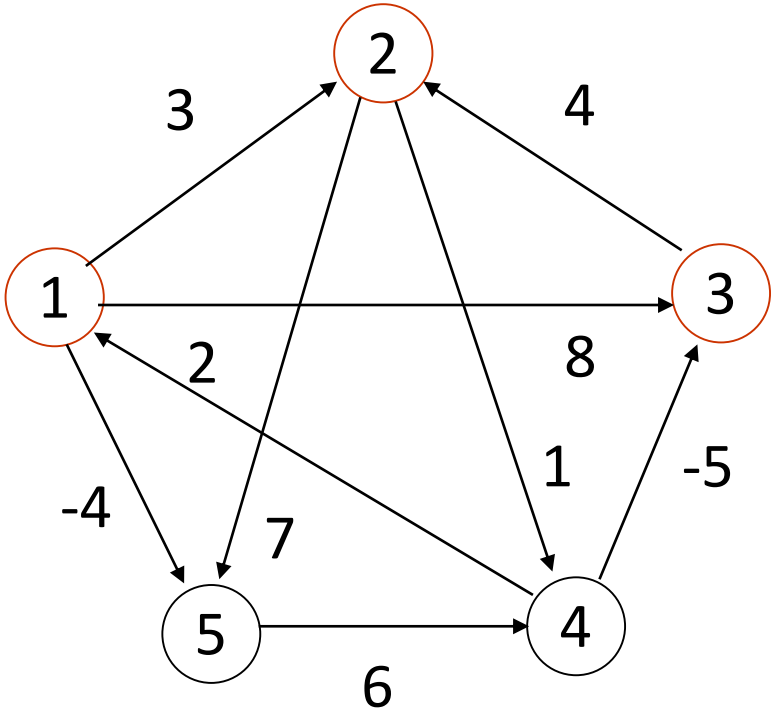
	1	2	3	4	5
1	0	3	8	4	-4
2	--	0	--	1	7
3	--	4	0	5	11
4	2	5	-5	0	-2
5	--	--	--	6	0



Floyd -Warshall Algorithm

δ^3

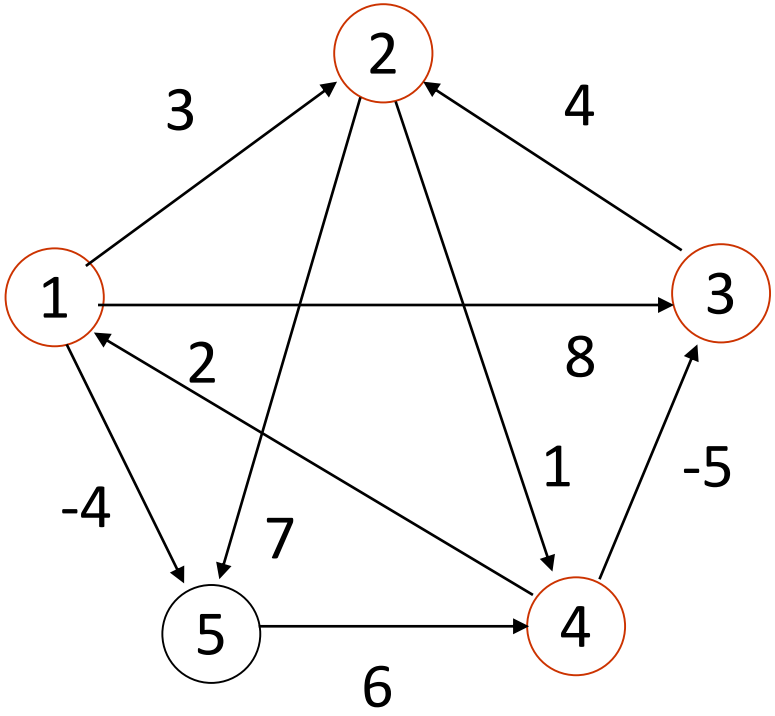
	1	2	3	4	5
1	0	3	8	4	-4
2	--	0	--	1	7
3	--	4	0	5	11
4	2	-1	-5	0	-2
5	--	--	--	6	0



Floyd -Warshall Algorithm

δ^4

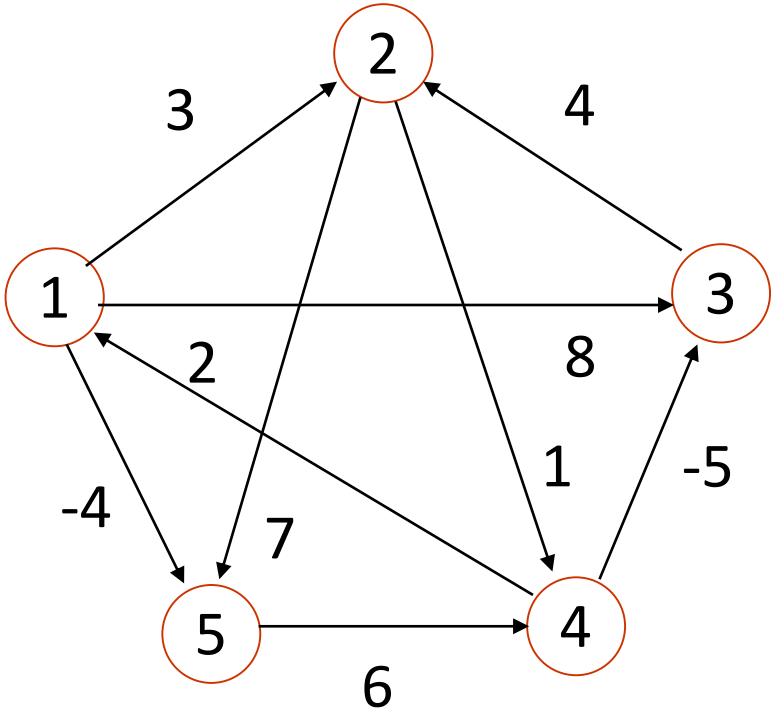
	1	2	3	4	5
1	0	3	-1	4	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0



Floyd -Warshall Algorithm

δ^5

	1	2	3	4	5
1	0	1	-3	2	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0



Floyd -Warshall Algorithm

$$\delta^k(i, j) \leftarrow \text{Min} \{ \delta^{k-1}(i, j), \delta^{k-1}(i, k) + \delta^{k-1}(k, j)\}.$$

Theorem: The value of $\delta^n(i, j)$ is the shortest path from i to j

Proof idea: By induction on k, the value of $\delta^k(i, j)$ is correct. In particular, for $\delta^n(i, j)$ we get the shortest path.

Floyd -Warshall Algorithm

Time complexity:

Each application of step 3 requires $O(n^2)$ operations, and step 3 is repeated n times. Thus, the total time complexity is $O(n^3)$.

Negative cycles detection:

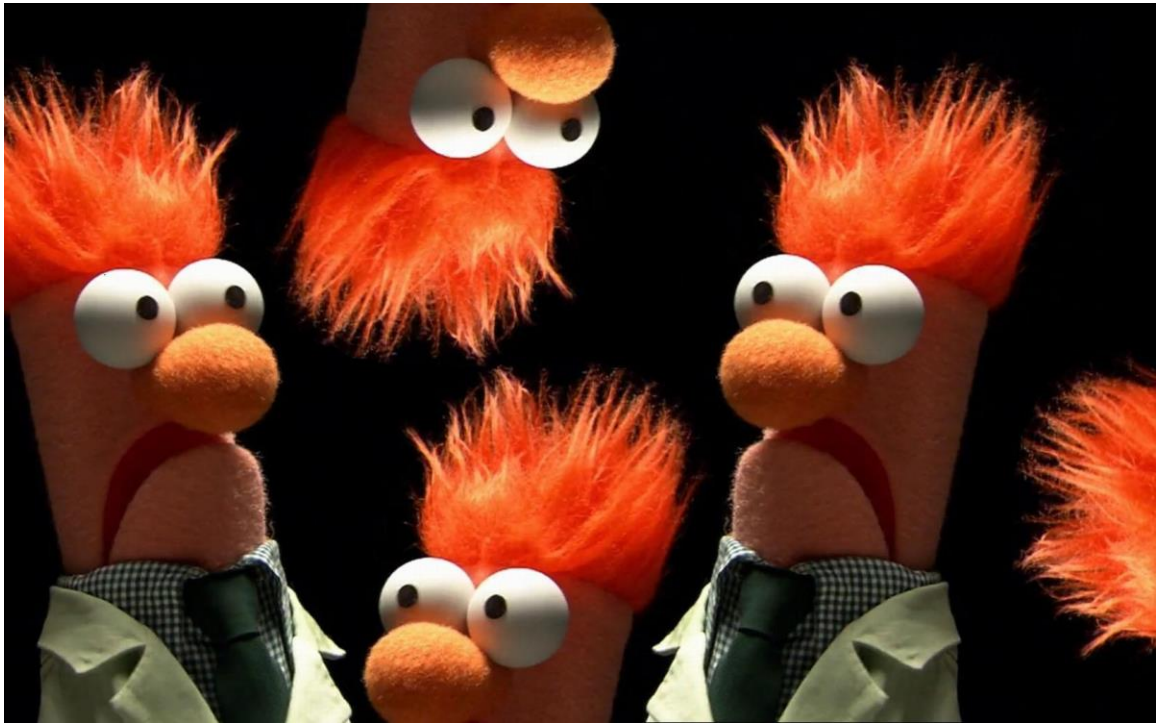
If a negative cycle exists, then for some i and k, we will get $\delta^k(i, i) < 0$.

Shortest-path algorithms - Summary

- Single source, no weights: BFS - O(n+m)
- Single source – DAG: O(n+m)
- Single source, non-negative weights: Dijkstra O((n + m) log n)) or $O(n^2)$ or (m+n log n).
- Single source, arbitrary weights:

  Bellman-Ford: O(nm)
- All-pair shortest path, arbitrary weights: Floyd-Warshall : $O(n^3)$

Mid-term Review



פרטים על הבוחן:

- יום שישי 9:00

אלגוריתמים – בוחן אמצע סמסטר.

שאלה	ניקוד
1	/32
2	/40
3	/28
ס"ה	/100

פרופ' תמי תמיר

5.12.25

מספר סטודנט:_____

משך המבחן: 100 דקות

- מותר דף עזר אחד.
- מותר להשתמש בדפי טיוטה אך יש לענות רק על דפי הבחינה, במקום המוקצב לכך בכל שאלה.
- רק הצד הקידמי בכל דף נסרק ומועבר לבדיקה.
- מותר לצטט ולהסתמך (מבלי להוכיח) על משפטים וטענות שהוכחו בכתה או בתרגילי הבית.
- בכל שאלה או סעיף שמופיע לידם סימן ♣, ניתן לכתוב 'לא יודע' ולקבל רבע מהניקוד (ערך תחתון).

בהצלחה!



Mid-term Details:

Friday 9:00

Algorithms. mid-term exam

Prof. Tami Tamir

5.12.25

Student I.D: _____

Question	
1	/32
2	/40
3	/28
Total	/100



Exam length: 100 minutes

- One reference page is allowed.
- You can use draft papers, but you need to answer only on the exam booklet, in the provided space and only on the front pages. Be aware that the back side is not scanned.
- You can cite and rely (without providing a proof) on theorems and claims that were proven in class or in the homework.
- In every question or section marked by ♣, you can write "I don't know" and get one quarter of the score (rounded down).

G O O D L U C K!

טכניקות מקובלות בבעיות 'הציעו אלגוריתם' למסלולים קצרים ביותר

1. שבפול הגרף, לכל עותק של הגרף מתאימה תכונה שמקיימים כל המסלולים שמגיעים אליו.
2. הוספת צמתים, שינוי כיווני הקשתות.
3. פירוק המסלול לקטעים שמקיימים תכונות מסוימות.

Common techniques in "Suggest an algorithm" problems for shortest path variants.

1. Graph Replication, each copy corresponds to some property of the paths reaching the copy.
2. Nodes addition, edge reversing – changes in the graph.
3. Partition a path into subpaths fulfilling some properties.

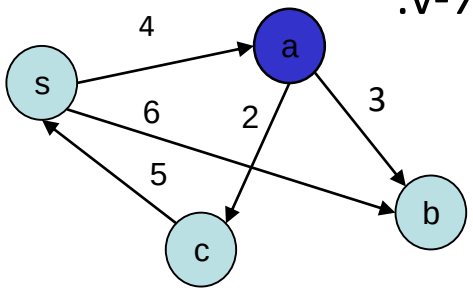


1. בעיית תחנות הדלק

נתון גרף מכוון $G=(V,E)$ עם משקלים חיוביים על הקשתות $w:E\rightarrow R^+$.
נתונה קבוצת צמתים $U\subseteq V$ וצומת מקור $s\in V$. נתון ש-$s\notin U$.

לכל צומת $v\in V$ נסמן ב-$d_U(s,v)$ את אורך מסלול קצר ביותר מ-s ל- v, שעובר בלפחות צומת אחד מהקבוצה U.
סיפור מתאים: בכל צומת $v\in U$ יש תחנת דלק, ועלינו לעבור בתחנת דלק בדרך מ-s ל-v.

Example: In the given graph, for U={a},
$d_U(s,s)=4+2+5=11$, $d_U(s,a)=4$, $d_U(s,b)=7$, $d_U(s,c)=6$



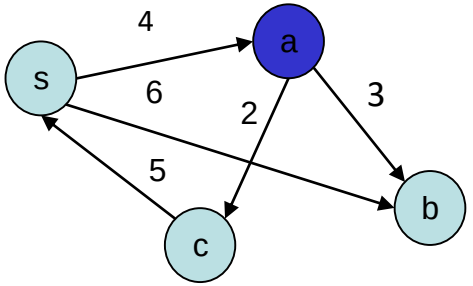
הבהרה: אם $v\in U$ אז אין צורך לעבור בצומת נוסף מ-U לפני ההגעה ל-v.

הציעו אלגוריתם יעיל שבהנתן G,U,s, מחשב לכל צומת $v\in V$ את הערך $d_U(s,v)$.
תארו את האלגוריתם, נמקו את נכונותו ונתחו את סיבוכיות זמן הריצה שלו.

1. The Gas Stations Problem.



Let $G=(V,E)$ be a directed graph with positive edge weights $w:E\to R^+$.
Let $s\in V$ and $U\subseteq V$. It is given that $s\notin U$
For every $v\in V$ denote by $d_U(s,v)$ the length of the shortest path from s to v that visits at least one vertex from U.
Application: In every $v\in U$ there is a gas station, and you must pass a gas station in your way.



Example: In the given graph, for U={a},
$d_U(s,s)=4+2+5=11$, $d_U(s,a)=4$, $d_U(s,b)=7$, $d_U(s,c)=6$

Clarification: If $v\in U$ then it is not required to visit another vertex from U before v.

Suggest an efficient algorithm for calculating, given G,U,s, the value of $d_U(s,v)$ for every $v\in V$. Describe the algorithm, justify its correctness, specify and justify its time complexity.

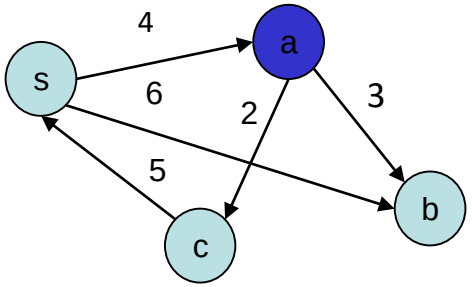
The Gas Stations Problem.

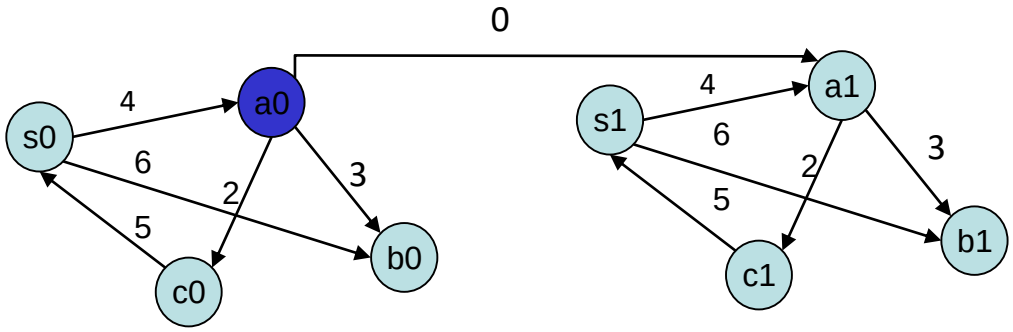
Algorithm:

1. Build a directed graph $G'=(V', E')$, where
$V'= V_0 \cup V_1$ (two copies of V). Denote by v_i the vertex v in copy i.
E': For every $(u,v) \in E$, E' includes the edges (u_0,v_0) and (u_1,v_1),
with $w(u_0,v_0)= w(u_1,v_1)= w(u,v)$.
Also, for every $u \in U$, E' includes the edge (u_0, u_1), with $w(u_0,u_1)=0$.

2. Run Dijkstra alg. on G', with s_0 as source.

3. For all $v \in V$ return $d_U(s,v)=d(s_0, v_1)$.





The Gas Stations Problem.

Correctness:

Idea: The graphs induced by V_0 and V_1 are copies of G. A transition from V_0 to V_1 is only via edges outgoing from a vertex in U, therefore, a path from s_0 to any node in V_1 includes an edge that leaves a node from U, thus, such a path includes at least one visit in U.

Main claim:

- G' includes a path of length k from s_0 to v_1 iff G includes a path of length k from s to v that visits at least one vertex from U.

In particular, the correctness of Dijkstra implies that $d(s_0, v_1)$ is the length of a shortest path that includes at least one node from U.

The Gas Stations Problem.

Time Complexity:

Let us analyze each step of the algorithm

1. $|V'|=2|V|$, $E'=2|E|+|W|$, Building G' takes $\Theta(|V'|+|E'|) = \Theta(|E|+|V|)$
2. Running Dijkstra on G': $\Theta((|V'| + |E'|) \log |V'|)= \Theta(|V|+|E|)\log|V|)$. (assuming implementation with binary heap)
3. Output: $\Theta(|V|)$

All together: $\Theta(|V|+|E|)\log|V|)$.

General structure of a solution based on copies of the graph

Algorithm:

1. Given G=(V,E), build G'=(V',E') and define w:E'→R

   V' – some function of V,E

   E' – some function of V,E.

2. Run some algorithm on G' (refer to G' when describing this run, for example, the source should be a node in V').
3. Conclude something about G from the run on G'.

General structure of a solution based on copies of the graph

Correctness:
There is a path with property X in G iff there is a path with property y in G'.

In particular: A shortest path with property X corresponds to a shortest path with property Y in G' (be careful here, sometimes the properties are involved).

What should be included in 'property'?

- The source and target of the path, and its weight.
- The characterization we want to keep track of.

Example: In G, there is a path from s to v, of length k, that has an even number of edges iff in G' there is a path from s_0 to v_2 of length k.

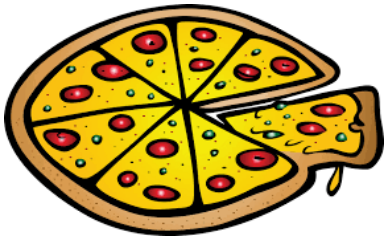
General structure of a solution based on copies of the graph

Time Complexity:

1. How much does it cost to build G'? What are |V'| and |E'|?
2. How much time it takes to run on G'? Specify it as a function of |V'| and |E'| and only then as a function of |V| and |E|.
3. How much time it takes to conclude the output?

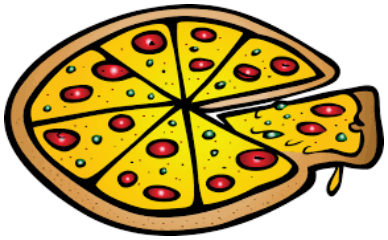
The time complexity should be given as a function of the input size (|V| and |E|).

2. Dijkstra Pizza



Given is a directed graph $G=(V,E)$ with positive weights on the edges. The graph represents a geographical map, and edge weights correspond to length of roads.
Also given is a set of nodes $W \subseteq V$. In every node $v \in W$ there is a branch of Dijkstra Pizza. Suggest an efficient algorithm for finding, for every $v \in V$, what is the minimal distance from v to the closest branch of Dijkstra Pizza.
Prove the correctness of your algorithm, specify and justify the time complexity.
Remarks:
1. For any $v \in W$, the output should be 0.
2. There exists an algorithm whose time complexity is equal to the time complexity of Dijkstra alg.

2. Dijkstra Pizza

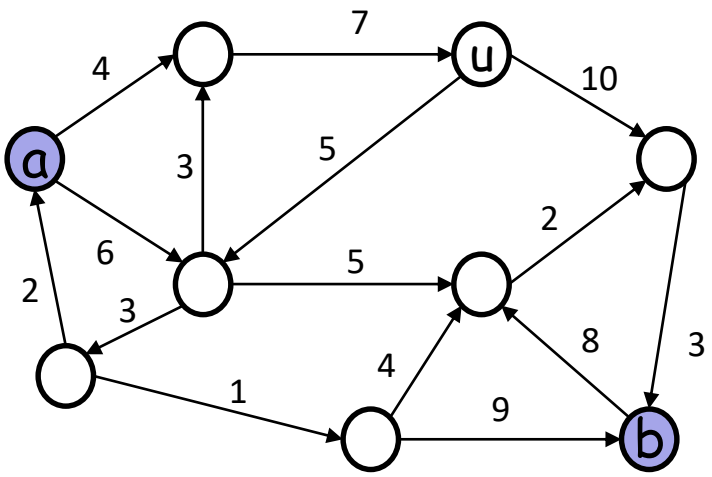


נתון גרף מכוון $G=(V,E)$, עם משקלים חיוביים על הקשתות.
הגרף מתאר מפת כבישים, ומשקל הקשתות מתאים לאורך הכביש המחבר את שני הצמתים.
נתונה קבוצת צמתים $W\subseteq V$, בכל צומת $v\in W$ יש סניף של Dijkstra Pizza.
הציעו אלגוריתם יעיל המוצא לכל צומת $v\in V$ מהו המרחק הקצר ביותר מ- v לסניף כלשהו של Dijkstra Pizza. הוכיחו את נכונות האלגוריתם ונתחו את סיבוכיותו.

הבהרות: 1. לכל $v\in W$, הערך המבוקש הוא אפס.
2. קיים פתרון שסיבוכיות הזמן שלו היא כסיבוכיות הזמן של אלג' Dijkstra.

Example:

W=(a,b)



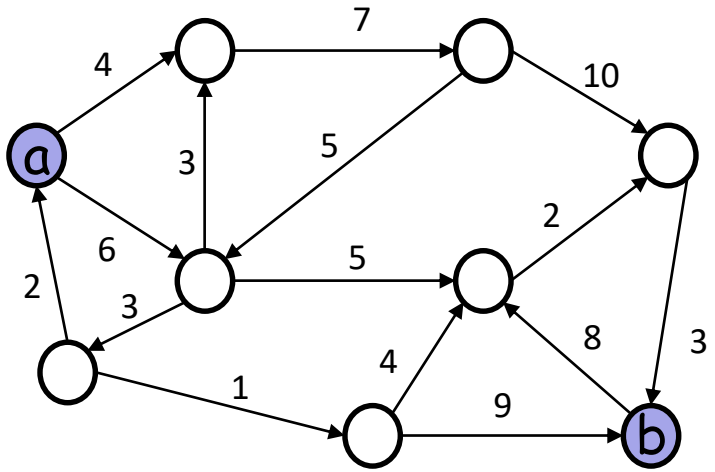
d(u,a)=10

d(u,b)=13,

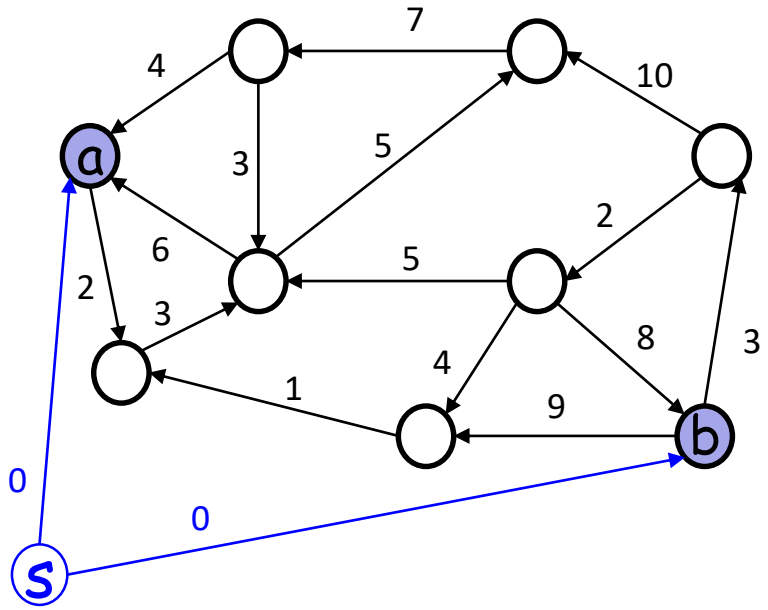
So for u, we should return 10.

Example:

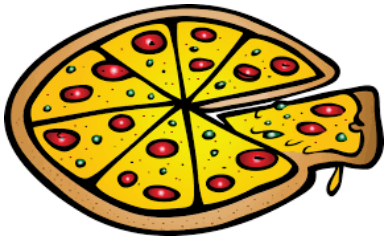
W=(a,b)



G’:



Dijkstra Pizza



Algorithm:

1. Given G=(V,E), build a graph G'=(V', E'):
   V' = V $\cup$ {s}.
   E'= {(s,v)| v$\in$W} $\cup$ {(u,v)| (v,u) $\in$E}.
   Edges of type (s,v) have weight 0. The reversed edges have the same weight as their corresponding edge in G.

In words: to construct G' from G, add a vertex s and connect it to all vertices of W with new 0-weight edges; change the direction of all edges of G.

2. Run Dijkstra algorithm on G', with s as the source.
3. The shortest path from v$\in$V to a branch of Dijkstra Pizza is d(s,v).

Correctness:

Claim: for every $v \in V$, there is a path of length k from v to some $w \in W$ in G **iff** there is a path of length k from s to v in G'.

Proof: (i) Let p be a path of length k from v to $w \in W$. The corresponding path from s in G' begins with the 0-weight edge (s,w), followed by the reverse of p. By the construction, such a path exists and has the same weight as p.
(ii) Let p be a path of length k from s to v in G'. It must begin with a zero-weight edge (s,w) to some $w \in W$ (as these are the only edges outgoing from s). The rest of the path corresponds to a reversed path from v to w in G, thus its total weight is k.

In particular, by the above claim, the shortest path from s to v in G' corresponds to the length of a shortest path from v to some node $w \in W$ in G.

Complexity:

$|V'|=|V|+1$, $|E'|=|E|+|W|\leq|E|+|V|$. Therefore, assuming implementation of Dijkstra with Fibonacci heaps,

Step 1 : $\Theta(|V|+|W|+|E|)= \Theta(|V|+|E|)$.

Step 2: $\Theta(|E'| + |V'|\log|V'|)= \Theta(|E|+|V|\log|V|)$.

Step 3: $\Theta(|V|)$.

Total: $\Theta(|E|+|V|\log|V|)$.

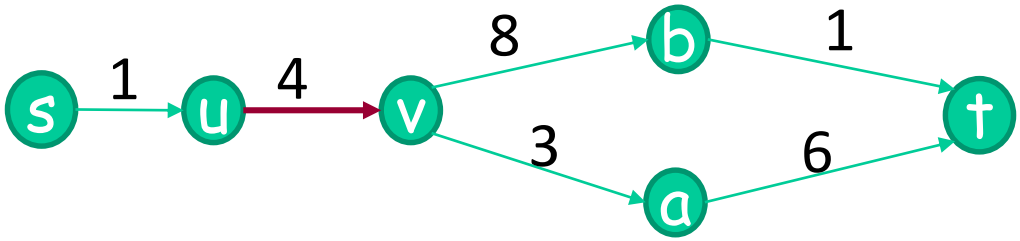
3. All or some SP

Let G be a directed graph with arbitrary edge weights, but no negative cycles.
Given two vertices $s,t \in V$ and an edge $(u,v) \in E$, suggest (two different) algorithms to determine whether:

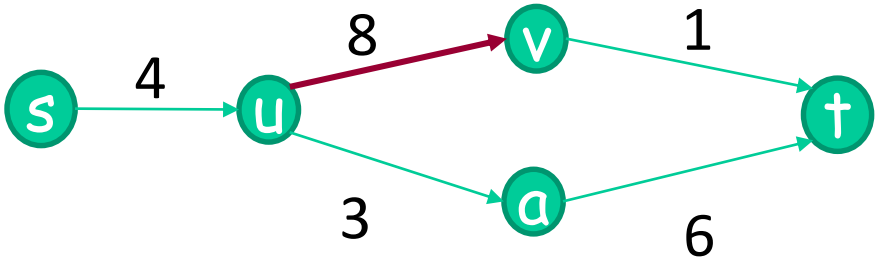
1. (u,v) belongs to **every** shortest path from s to t.
2. (u,v) belongs to **some** shortest path from s to t.

Explain shortly the correctness of your algorithms and analyze their time complexity.

Example:



The edge (u,v) is on every SP from s to t



The edge (u,v) is on some SP from s to t

Note: If an edge is on every SP from s to t, then clearly, it is also on some SP from s to t.

Solution Is (u,v) on every s-t SP?

1. Run Bellman-Ford on G, calculate d(s,t).
2. Let G' be a graph produced from G by increasing the weight of (u,v) [by any constant, in particular, it is also OK to remove the edge (u,v)].
3. Run Bellman-Ford on G', calculate d'(s,t).
4. (u,v) belongs to **every** shortest path from s to t if and only if d'(s,t)>d(s,t).

Explanation: The shortest path is hurt if and only if there is no way to achieve d(s,t) without passing through (u,v). That is, there is no way to avoid the increase in (u,v)'s weight.
Time Complexity: Dominated by running BF twice; $\Theta(|V||E|)$.

Solution Is (u,v) on some s-t SP?

1. Run Bellman-Ford on G with s as source, calculate d(s,t), d(s,u).
2. Run Bellman-Ford on G with v as source, calculate d(v,t).
3. (u,v) belongs to **some** shortest path from s to t if and only if d(s,t)=d(s,u)+w(u,v)+d(v,t).

Explanation: We check whether the shortest path is hurt if the path is forced to pass through (u,v). The shortest path that is forced to pass through (u,v) consists of three parts: the shortest path from s to u, the edge (u,v), and the shortest path from v to t.

Time Complexity: Dominated by running BF twice; O(|V||E|).

Solution Is (u,v) on some s-t SP?

Remark: An alternative solution of decreasing (u,v)'s weight and checking whether d(s,t) decreases may cause a construction of a negative cycle, and is therefore wrong.